# Red Hat Ceph Storage NFS cluster and share management (OpenStack Manila users only)

Create and manage NFS file services, including clusters, shares, QoS, security, monitoring, backup and restore, and troubleshooting.

Create and manage NFS services for Ceph File Systems. This content includes procedures for cluster and share management, QoS configuration, security features such as Kerberos, TLS, and RDMA, monitoring, backup and restore, and troubleshooting.

**Important:** The NFS gateway in Red Hat Ceph Storage is supported only when used with Red Hat OpenStack Services on OpenShift (RHOSO). Standalone use of the Ceph NFS service is not supported.

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- Installation and configuration of the Ceph Metadata Server daemons (`ceph-mds`).
- Create and mount a Ceph File System.

**Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

## Creating an NFS cluster

Create an NFS cluster with the **`ceph nfs cluster create`** command. This creates a common recovery pool for all NFS service daemons, new user based on the cluster name, and a common NFS service configuration RADOS object.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- An existing Ceph File System.
- Root-level access to Ceph Monitor.
- Installation of the `nfs-pool`, `nfs-pool-ceph`, `nfspool-rados-grace`, and `nfs-pool-rados-urls` packages on the Ceph Manager hosts.
- Root-level access to the client.

**Note:** NFS cluster creation can currently be done only with the command-line interface. Creating NFS clusters through the dashboard is not currently supported.

1. Log into the `cephadm` shell.
   For example,

   ```
   [root@mds ~]# cephadm shell
   ```

2. Enable the Ceph Manager NFS module.

   ```
   ceph mgr module enable nfs
   ```

   For example,

```
[ceph: root@host01 /]# ceph mgr module enable nfs
```

3. Create an NFS service cluster.

```
ceph nfs cluster create CLUSTER_NAME [PLACEMENT] [--ingress] [--virtual_ip IP_ADDRESS]
[--ingress-mode {default|keepalive-only|haproxy-standard|haproxy-protocol}] [--port
PORT]
```

> **Important:** Red Hat only supports one NFS service daemon running per host.

For example,

```
[ceph: root@host01 /]# ceph nfs cluster create nfs-cephfs "host01 host02"
NFS Cluster Created Successfully
```

In this example, the NFS service cluster name is `nfs-cephfs` and the daemon containers are deployed to `host01`, and `host02`.

4. Verify the NFS service cluster information.

```
ceph nfs cluster info [CLUSTER_NAME]
```

> **Note:** Specifying the *CLUSTER_NAME* is optional.

For example,

```
[ceph: root@host01 /]# ceph nfs cluster info {
"nfs-cephfs": {
"backend": [
{
"hostname": "host01",
"ip": "10.8.129.4",
"port": 2049
},
{
"hostname": "host02",
"ip": "10.8.129.4",
"port": 2049
}
],
"virtual_ip": null
}
}
```

## Customize an NFS configuration

Customize a Ceph-managed NFS cluster by supplying a user configuration file. The user configuration is stored and managed by `cephadm` and is merged with the generated defaults at runtime. The user configuration takes precedence only for supported parameters.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Metadata Server (MDS) node.
- An NFS cluster created using the `ceph nfs cluster create` command.

Use this procedure to add or update supported NFS settings, such as logging, export defaults, and selected TLS policies for an existing NFS cluster. The configuration is stored by Ceph (in RADOS) and overlaid at runtime on the container-generated base configuration.

1. Create a nfs-ganesha.conf file with custom parameters.

```
[root@ceph8-node1 ~]# cat ganesha2.conf
NFS_CORE_PARAM {
 Enable_NLM = true
 NFS_Protocols = 3,4;
NFS_Port = 2050;
 MNT_Port = 4048;
 mount_path_pseudo = true;
}
```

2. Apply the custom configuration using below command.

```
$ ceph nfs cluster config set singletest -i PATH_TO_CONFIG_FILE
```

For example,

```
$ ceph nfs cluster config set nfs-cephfs -i ganesha2.conf
```

3. Get the custom config, check port in netstat and check Ganesha.conf inside container.

```
[root@ceph8-node1 ~]# ceph nfs cluster config get singletest
NFS_CORE_PARAM {
 Enable_NLM = true
 NFS_Protocols = 3,4;
NFS_Port = 2050;
 MNT_Port = 4048;
 mount_path_pseudo = true;
}
[root@ceph8-node1 ~]# netstat -tnlp | grep -i nfs
tcp6    0   0 :::2049         :::*          LISTEN   1476200/ganesha.nfs
tcp6    0   0 :::50051        :::*          LISTEN   1476200/ganesha.nfs
tcp6    0   0 :::9587         :::*          LISTEN   1476200/ganesha.nfs
tcp6    0   0 :::46089        :::*          LISTEN   1476200/ganesha.nfs
[root@ceph8-node1 ~]# cephadm shell
[ceph: root@ceph8-node1 /]# /bin/rados -p .nfs -N singletest get userconf-
nfs.singletest -
NFS_CORE_PARAM {
 Enable_NLM = true
 NFS_Protocols = 3,4;
NFS_Port = 2050;
 MNT_Port = 4048;
 mount_path_pseudo = true;
}
```

*NFS customization parameters*

| Parameter | Default/Value | Description |
|---|---|---|
| Enable_NLM | true | Enables Network Lock Manager to support NFSv3 locking semantics; set to true when NFSv3 clients require file locks. |
| NFS_Protocols | 3,4 | Specifies the NFS protocol versions the server supports and listens for. |
| NFS_Port | 2050 | Intended TCP service port for NFS. In Ceph-managed NFS, port binding is controlled at the service layer (for example, during cluster creation with a service port) and this overlay value does not change the listening socket. |
| MNT_Port | 4048 | Intended mountd port. Not applied via the user configuration overlay when the service is managed by cephadm. |
| mount_path_pseudo | true | Enables pseudo-path mounting for NFSv3, aligning its behavior with NFSv4-style pseudo roots. |

## Result

- The NFS cluster applies the user-defined configuration successfully, and the updated settings can be verified through the **`ceph nfs cluster config get`** command and functional validation.

## Remove customized NFS configurations

Remove previously applied user-defined NFS configuration overrides and restore the NFS cluster to its default configuration blocks.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A running and healthy Red Hat Ceph Storage cluster.
- The NFS cluster name.
- You have administrative access to run Ceph CLI commands.

The reset commands removes all user-defined parameters that were previously applied with the **`ceph nfs cluster config set`** command. The reset operation reverts the NFS service to its default cephadm-managed configuration and removes the related user configuration object from the `.nfs` pool.

1. Remove the user-defined configuration.

   ```
   ceph nfs cluster config reset CLUSTER_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph nfs cluster config reset nfs-cephfs
   ```

2. Optional: Verify that the configuration was removed.

   ```
   ceph nfs cluster config get CLUSTER_NAME
   ```

   > **Note:** If no user configuration is present, the command returns either an empty result or only the default values managed by cephadm.

### Result

The user-defined NFS configuration is removed, and the cluster reverts to the default cephadm-generated settings. The verification command confirm that no custom configuration remains, and the service operates using only the default configuration blocks.

# Sharing Ceph File System namespaces over the NFS protocol

Ceph File Systems (CephFS) namespaces can be shares over the NFS protocol using a NFS service file server. To share a CephFS namespace, you must first have a running NFS service cluster.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- An NFS cluster created using the **`ceph nfs cluster create`** command.

> **Important:**
>
> - NFSv3 (version 3) is supported and enabled for clients that do not yet support NFSv4 mounts for CephFS file shares.
>   Use NFSv4 for clients and servers that support NFSv4.2 or later.

- NFS clients are unable to create CephFS snapshots through their native NFS mount. They must use server-side operator tooling for their snapshot needs.

1. Create a CephFS share.

   **Important:** Due to a known issue, do not use the **cmount_path** option when creating an NFS share. If a value other than / is used, NFS shares that were previously defined with the / path become inaccessible.

   ```
   ceph nfs export create cephfs CLUSTER_NAME BINDING FILE_SYSTEM_NAME [--readonly] [--
   path=PATH_WITHIN_CEPHFS]
   ```

   In the following example, the *BINDING* (/ceph) is the pseudo root path, which must be unique, and an absolute path.

   ```
   [ceph: root@host01 /]# ceph nfs export create cephfs nfs-cephfs /ceph cephfs01 --
   path=/
   {
   "bind": "/ceph",
   "cluster": "nfs-cephfs",
   "fs": "cephfs01",
   "mode": "RW",
   "path": "/"
   }
   ```

   **Note:**

   - The **--readonly** option shares a path with the read-only permission, the default being read, and write permissions.
   - The **PATH_WITHIN_CEPHFS** can be a subvolume. You can get the absolute subvolume path by using the **subvolume getpath** command.

     ```
     ceph fs subvolume getpath VOLUME_NAME SUBVOLUME_NAME [--
     group_name SUBVOLUME_GROUP_NAME]
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph fs subvolume getpath cephfs sub0
     ```

2. View the share block based on the pseudo root name.

   ```
   ceph nfs export get CLUSTER_NAME BINDING
   ```

   For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

3. List the NFS shares.

```
ceph nfs export ls CLUSTER_NAME [--detailed]
```

For example,

```
[ceph: root@host01 /]# ceph nfs export ls nfs-cephfs --detailed
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/service1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

4. Get the information of the NFS share.

```
ceph nfs export info CLUSTER_NAME [PSEUDO_PATH]
```

For example,

```
[ceph: root@host01 /]# ceph nfs export info nfs-cephfs /ceph
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

5. On a client host, mount the shared Ceph File System.

```
mount -t nfs -o port=GANESHA_PORT HOST_NAME:BINDING LOCAL_MOUNT_POINT
```

For example,

```
[root@client01 ~]# mount -t nfs -o port=2049 host01:/ceph/ /mnt/nfs-cephfs
```

a. To automatically mount on boot, open and edit the `/etc/fstab` file by adding a new line.

```
HOST_NAME:BINDING LOCAL_MOUNT_POINT nfs4
defaults,seclabel,vers=4.2,proto=tcp,port=2049  0  0
```

> **Note:** For NFSv3 mounts, use **`vers`** = 3.

For example,

```
[root@client01 ~]# host01:/ceph/   /mnt/nfs-cephfs   nfs4
defaults,seclabel,vers=4.2,proto=tcp,port=2049  0  0
```

6. On a client host, mount a shared NFS Ceph File System, created with an `ingress` service.

```
mount -t nfs VIRTUAL_IP_ADDRESS:BINDING LOCAL_MOUNT_POINT
```

- Replace *VIRTUAL_IP_ADDRESS* with **`--ingress --virtual-ip`** IP address used to create the NFS cluster.
- Replace *BINDING* with the pseudo root path.
- Replace *LOCAL_MOUNT_POINT* with the mount point to mount the share on.

The following example mounts the share `nfs-cephfs` that exists on a NFS cluster created with `--ingress --virtual-ip 10.10.128.75` on the `/mnt` mount point .

```
[root@client01 ~]# mount -t nfs 10.10.128.75:/nfs-cephfs /mnt
```

## Modifying NFS shares

Modify Ceph File System share parameters with a configuration file.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- An NFS share created.

You can modify the following parameters in a share with a configuration file:

**`access_type`**

This can be `RW`, `RO`, or `NONE`.

**`squash`**

This can be `No_Root_Squash`, `None`, or `Root_Squash`.

**`security_label`**

This can be `true` or `false`.

> **Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

1. View the share block based on the pseudo-root name.

```
ceph nfs export get CLUSTER_NAME BINDING
```

For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

2. Share the configuration file.

```
ceph nfs export get nfs-cephfs /ceph > export.conf
```

For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph > export.conf
```

3. Edit the share information.

> **Note:** For NFSv3 mounts (NFS version 3), use **`protocols`** = 3.

```
{
  "export_id": EXPORT_ID,
  "path": "PATH",
  "cluster_id": "CLUSTER_NAME",
  "pseudo": "CLUSTER_PSEUDO_PATH",
  "access_type": "RW/RO",
  "squash": "SQUASH",
  "security_label": SECURITY_LABEL,
  "protocols": [
    PROTOCOL_ID
  ],
  "transports": [
    "TCP"
  ],
  "fsal": {
    "name": "NAME",
    "user_id": "USER_ID",
    "fs_name": "FILE_SYSTEM_NAME",
    "sec_label_xattr": ""
  },
  "clients": []
}
```

In the following example, the `access_type` is modified from `RO` to `RW`.

```
[ceph: root@host01 /]# vi export.conf

{
"access_type": "RW",
"clients": [],
"cluster_id": "nfservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

4. Apply the specification.

```
ceph nfs export apply CLUSTER_NAME PATH_TO_EXPORT_FILE
```

For example,

```
[ceph: root@host01 /]# ceph nfs export apply nfs-cephfs -i export.conf
Added export /ceph
```

5. Get the updated share information.

```
ceph nfs export get CLUSTER_NAME BINDING
```

For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsganesha.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

## Creating custom NFS shares

You can customize the Ceph File System (CepFS) shares and apply the configuration.

> **Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

## Prerequisites

- A running, and healthy Red Hat Ceph Storage cluster.
- An NFS cluster created using the `ceph nfs cluster create` command.
- A CephFS created.

## Procedure

1. Create a custom file:

   Example

   ```
   [ceph: root@host01 /]# touch export_new.conf
   ```

2. Create a share using the custom file:

   Syntax

   > **Note:** For NFSv3 mounts, use 3 for the **`Protocols`** parameter.

   ```
   EXPORT {
     Export_Id = EXPORT_ID;
     Transports = TCP/UDP;
     Path = PATH;
     Pseudo = PSEUDO_PATH;
     Protocols = NFS_PROTOCOLS;
     Access_Type = ACCESS_TYPE;
     Attr_Expiration_Time = EXPIRATION_TIME;
     Squash = SQUASH;
     FSAL {
       Name = NAME;
       Filesystem = "CEPH_FILE_SYSTEM_NAME";
       User_Id = "USER_ID";
     }
   }
   ```

   Example

   ```
   [ceph: root@host01 /]# cat export_new.conf
   EXPORT {
     Export_Id = 2;
     Transports = TCP;
     Path = /;
     Pseudo = /ceph1/;
     Protocols = 4;
     Access_Type = RW;
     Attr_Expiration_Time = 0;
     Squash = None;
     FSAL {
       Name = CEPH;
       Filesystem = "cephfs";
       User_Id = "nfs.nfs-cephfs.2";
     }
   }
   ```

3. Apply the specification:

   Syntax

   ```
   ceph nfs export apply CLUSTER_NAME -i PATH_TO_EXPORT_FILE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph nfs export apply nfs-cephfs -i new_export.conf
   Added export /ceph1
   ```

4. Get the updated share information:

Syntax

```
ceph nfs export get CLUSTER_NAME BINDING
```

For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph1
{
  "export_id": 1,
  "path": "/",
  "cluster_id": "nfs-cephfs",
  "pseudo": "/ceph1",
  "access_type": "RW",
  "squash": "None",
  "security_label": true,
  "protocols": [
    4
  ],
  "transports": [
    "TCP"
  ],
  "fsal": {
    "name": "CEPH",
    "user_id": "nfs.nfs-cephfs.2",
    "fs_name": "cephfs",
    "sec_label_xattr": ""
  },
  "clients": []
}
```

## Deleting NFS shares from the CLI

You can delete the Ceph File System (CephFS) NFS shares with the **ceph export rm** command.

- Delete an NFS share.

  ```
  ceph nfs export rm CLUSTER_NAME BINDING
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph nfs export rm nfs-cephfs /ceph
  ```

## Deleting an NFS cluster

Delete an NFS cluster with the **nfs cluster rm** command. This deletes the deployed cluster. The removal of NFS daemons and the ingress service is asynchronous. Check the status of removal with the **ceph orch ls** command.

### Before you begin

- A running, and healthy Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Metadata Server (MDS) node.
- NFS daemons deployed with **ceph nfs cluster create** command.

### Procedure

1. Log into the `cephadm` shell.

   For example,

   ```
   [root@mds ~]# cephadm shell
   ```

2. Remove the NFS cluster.

   ```
   ceph nfs cluster rm CLUSTER_NAME
   ```

For example,

```
[ceph: root@host01 /]# ceph nfs cluster rm nfs-cephfs
```

## Windows 11 desktop client side mount options for NFSv3 fileshare

Use this information to understand the client side mount options for NFSv3 fileshare.

NFSv3 (version 3) is supported and enabled for clients that do not yet support NFSv4.2 mounts for CephFS file shares.

> **Note:** Mount a file share with NFSv3 Windows 11 desktop only when NFSv4.2 is not supported on the client or server side. For more information about configuring NFS, see Configuring NFS Service daemons.

Use the following mount client-side options to mount a file share with NFSv3 Windows 11 desktop:

```
Usage:  mount [-o options] [-u:username] [-p:<password | *>] <\\computername\sharename>
<devicename | *>
```

*Windows 11 mount options for NFSv3*

| Mount options | Description |
|---|---|
| **-o rsize=size** | Set the size of the read buffer in kilobytes. |
| **-o wsize=size** | Set the size of the write buffer in kilobytes. |
| **-o timeout=time** | Set the timeout value in seconds for an RPC call. |
| **-o retry=number** | Set the number of retries for a soft mount. |
| **-o mtype=soft\|hard** | Set the mount type. |
| **-o lang=euc-jp\|euc-tw\|euc-kr\|shift-jis\|big5\|ksc5601\|gb2312-80\|ansi** | Specify the encoding used for file and directory names. |
| **-o fileaccess=mode** | Specify the permission mode of the file. These are used for new files created on NFS servers. Specified using UNIX style mode bits. |
| **-o nolock** | Disable locking. |
| **-o anon** | Mount as an anonymous user. |
| **-o casesensitive=yes\|no** | Specify case sensitivity of file lookup on server. |
| **-o sec=sys\|krb5\|krb5i\|krb5p** | Specify the security mechanism for mounting an NFS share. This value can include any of the following:<br>– **sec=sys:** No authentication or security checks are performed. Data transfers are not encrypted.<br>– **sec=krb5:** Specifies Kerberos authentication for mounting the NFS share.<br>– **sec=krb5i:** Extends Kerberos security by adding data integrity checks between the client and server hasn't been tampered.<br>– **sec=krb5p:** Enhances Kerberos security by enabling data privacy, which encrypts data exchanged between the client and server. |

## Rate-limiting in NFS

With the rate-limiting or QoS (quality of service) feature in NFS you can control the client's read/write bandwidth and rate-limiting of the client's input/output operations (IOPS). This function operates at share and client levels, controlling read/write requests and IOPS. QoS helps ensures that all clients accessing backend storage receives a fair share of resources.

You can enable QoS for a single node NFS server and multiple instances of NFS servers.

The QoS configuration must be stored in the RADOS object and be accessible by any NFS server within the server group. Also, each share must reside on only one NFS server within the group.

The QoS function in NFS supports both bandwidth control and IOPS control.

> **Note:** The rate-limiting feature is supported only for NFS version 4 and later.

## Bandwidth rate-limiting

NFS bandwidth rate limiting allows administrators to control the amount of bandwidth that NFS clients use when accessing or transferring data from an NFS server. This helps prevent excessive network congestion, ensures fair bandwidth distribution, and optimizes network performance for all users and applications.

The following are supported options in bandwidth rate-limiting:

**Per share *(export)* bandwidth control**

Use this option to configure the maximum bandwidth that is allowed per share *(export)*. In this case, QoS makes sure that the cumulative bandwidth of all clients accessing this share does not exceed the set bandwidth. For example, if the bandwidth limit for a share is set to 100 Mbps, any client running any operation on this share is given only 100 Mbps speed.

**Per client bandwidth control**

Use this option to configure the maximum bandwidth that is allowed per client. In this case, even if the client accesses more than one share, the total bandwidth that is given to the client will not exceed the set limit. For example, if the client bandwidth is set to 100 Mbps, this client can perform operations on any share with only 100 Mbps speed.

**Per share *(export)*-per client bandwidth control**

Use this option to configure both per share maximum bandwidth and per client maximum bandwidth. In this case, the maximum client bandwidth is controlled to the set limit, while the maximum per share bandwidth is also maintained.
For example, if the per share bandwidth is set to 200 Mbps and the bandwidth per client is set to 100 Mbps, client A writing to this share will have a speed of 100 Mbps. If client B writes to the same share, client B will have a rate limit of 100 Mbps, as 100 Mbps is already in use by client A.

In bandwidth rate-limiting, you can configure the maximum read and write bandwidth individually. For example, you can set the maximum write bandwidth to 100 Mbps and the read bandwidth to 200 Mbps.

## Input/output operations (IOPS) rate-limiting

IOPS rate-limiting is a feature that controls the number of input/output operations per second that NFS clients can run on an NFS server. This helps prevent performance degradation, ensures fair resource allocation, and optimizes storage system efficiency.

Read and write operations consist of multiple underlying tasks. For example, a write operation includes four tasks: get file, open file, write file, and put file. If you set the IOPS limit to 20, only 5 IOPS allowed for each of these tasks.

The following are supported options in IOPS rate-limiting are:

**Per share operations control**

Use this option to configure the maximum number of IOPS allowed per share. In this case, QoS makes sure that the cumulative number of operations that are performed by all clients accessing this share will not exceed the given limit. For example, if the IOPS limit is set to 20, then any client accessing this share can run only 5 IOPS.

**Per client operations control**

Use this option to configure the maximum number of operations that are allowed per client. Regardless of the number of shares that the client is accessing, the total number of operations that are performed by the client will not exceed this limit. For example, if the IOPS limit for a client is set to 20, then this client can perform only 5 IOPS, irrespective of the number of shares it is accessing.

**Per share-per client operations control**

Use this option to configure the maximum that is allowed per share and maximum that is allowed per client operations. The total number of operations that are run by the client is controlled to the set limit. Simultaneously, QoS helps ensure the maximum operations per share limit is kept within the limits. For example, if the IOPS limit for share is set to 20 and the client limit is set to 40, then this client can run only 5

operations on the share as its limit is set to 20. The client can run the remaining 3 operations on other shares.

For IOPS rate-limiting, all operations that the client runs are counted together as a single unit.

# Managing rate-limiting for NFS clusters

Learn to manage the rate-limiting or quality of service (QoS) for NFS clusters.

There are 2 types of rate-limiting for NFS clusters:

- Bandwidth rate-limiting.
- Input/output (IOPS) rate-limiting.

You can enable and disable the rate-limiting feature by using the command line.

## Enabling bandwidth rate-limiting for NFS cluster

Learn to enable the bandwidth rate-limiting feature by using the command line.

You can define bandwidth limits per client or share (per share) to ensure a balanced workload distribution across the cluster.

> **Note:** The bandwidth rate-limiting, the limit ranges from a minimum of 1 Mbps to a maximum of 4 Gbps.

**Understanding cluster-wide enforcement of bandwidth and IOPS limits**
When you enable bandwidth or input/output operations (IOPS) control on an NFS cluster, the configured limits apply cumulatively across all hosts in the cluster, rather than per individual host.

To enforce these cumulative limits, NFS services coordinate with NFS daemons running on other hosts in the cluster. This coordination ensures that bandwidth and IOPS limits are enforced consistently, even when multiple NFS servers handle client requests at the same time.

> **Note:** You can configure the port used for this inter-node communication by using the cluster_qos_port field in the NFS specification. The default port is 31311.

You can control how frequently NFS daemons exchange coordination messages by using the cqos_msg_interval field in the NFS specification. The default interval is 100 ms.

Alternatively, you can update the message interval by using the command-line interface.

```
ceph nfs cluster qos set <cluster_id> <msg_interval>
```

This inter-node coordination ensures accurate enforcement of bandwidth and IOPS limits across the NFS cluster.

**Cluster QoS TCP keepalive configuration**
Cluster QoS uses TCP connections between NFS nodes. To keep these TCP connections open, configure TCP keepalive parameters in the `NFS_CORE_PARAMS` section of the nfs-ganesha.conf file on all NFS nodes.

Configure the following parameters:

```
TCP_KEEPCNT = 5;
TCP_KEEPIDLE = 30;
TCP_KEEPINTVL = 2;
```

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.
- An NFS cluster.
- Run the following command to enable bandwidth rate-limiting for an NFS cluster.

```
ceph nfs cluster qos enable bandwidth_control CLUSTER ID qos_type:PerShare|PerClient|
PerShare_PerClient [--combined-rw-bw-ctrl] [--max_export_write_bw VALUE] [--
max_export_read_bw VALUE] [--max_client_write_bw VALUE] [--max_client_read_bw VALUE]
[--max_export_combined_bw VALUE] [--max_client_combined_bw VALUE]
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- **qos_type** is the type of bandwidth control. You can select either **PerShare** (per share), **PerClient** or **PerShare_PerClient** based on your requirement.
  - If the **PerShare** qos_type is selected, the cluster-level quality of service (QoS) configuration applies to all shares created on that NFS cluster. If **combined-rw-bw-ctrl** is set to false, you must specify both the **max_export_write_bw** and **max_export_read_bw** parameters. Otherwise, only the **max_export_combined_bw** parameter is required.
  - If the **PerClient** qos_type is selected, the cluster-level QoS configuration applies to all clients accessing shares on that cluster. If **combined-rw-bw-ctrl** is set to false, you must specify both the **max_client_write_bw** and **max_client_read_bw** parameters. Otherwise, only the **max_client_combined_bw** parameter is required
  - If the **PerShare_PerClient** qos_type is selected, the cluster-level configuration applies to all shares and all clients of that NFS cluster. If **combined-rw-bw-ctrl** is set to false, you must specify the **max_export_write_bw**, **max_export_read_bw**, **max_client_write_bw**, and **max_client_read_bw** parameters. Otherwise, only the **max_export_combined_bw** and **max_client_combined_bw** parameters are required.
- **combined-rw-bw-ctrl** This parameter enables combined read and write bandwidth. If **combined-rw-bw-ctrl** is set to true, then only **max_export_combined_bw** and **max_client_combined_bw** parameters are allowed based on QoS type.
- **max_export_write_bw** is the maximum bandwidth for share write per second.
- **max_export_read_bw** is the maximum bandwidth for share that is read per second.
- **max_client_write_bw** is the maximum bandwidth that the client can use for writing per second.
- **max_client_read_bw** is the maximum bandwidth that the client can use for reading per second.
- **max_export_combined_bw** is the maximum combined read/write bandwidth for share per second.
- **max_client_combined_bw** is the maximum combined read/write bandwidth for client per second.

> **Note:** You can specify the value as 10, 10 MB, 10 MiB, or 10 GB.

**Example of bandwidth rate-limiting for PerShare, clust_nfs is the NFS cluster ID**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerShare --max_export_write_bw 100MB --max_export_read_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_export_read_bw": "200.0MB",
  "max_export_write_bw": "100.0MB",
  "qos_type": "PerShare"
}
```

**Example of bandwidth rate-limiting for PerShare with combined read/write share limit**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerShare --combined-rw-bw-ctrl --max_export_combined_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": true,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_export_combined_bw": "200.0MB",
  "qos_type": "PerShare"
}
```

**Example of bandwidth rate-limiting for `PerClient`**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerClient --max_client_write_bw 100MB --max_client_read_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_client_read_bw": "200.0MB",
  "max_client_write_bw": "100.0MB",
  "qos_type": "PerClient"
}
```

**Example of bandwidth rate-limiting for `PerClient` with combined read/write share limit.**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerClient --combined-rw-bw-ctrl --max_client_combined_bw 300MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": true,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_client_combined_bw": "300.0MB",
  "qos_type": "PerClient"
}
```

**Example of bandwidth rate limiting for `PerShare_PerClient`**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerShare_PerClient --max_export_write_bw 100MB --max_export_read_bw 200MB --
max_client_write_bw 100MB --max_client_read_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_client_read_bw": "200.0MB",
  "max_client_write_bw": "100.0MB",
  "max_export_read_bw": "200.0MB",
  "max_export_write_bw": "100.0MB",
  "qos_type": "PerShare_PerClient"
}
```

**Example of bandwidth rate-limiting for `PerShare_PerClient` with combined read/write share limit**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerShare_PerClient --combined-rw-bw-ctrl --max_export_combined_bw 200MB --
max_client_combined_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": true,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_client_combined_bw": "200.0MB",
  "max_export_combined_bw": "200.0MB",
  "qos_type": "PerShare_PerClient"
}
```

By default, all shares inherit the cluster-level QoS settings.

> **Note:** If you use the command in the procedure to update the **qos_type**, make sure to update all the shares with the required parameters.

## Disabling bandwidth rate-limiting for NFS cluster

You can disable bandwidth rate-limiting by using the command line.

- Run the following command to disable bandwidth-rate limiting.

  ```
  ceph nfs cluster qos disable bandwidth_control CLUSTER_ID
  ```

  Where,
  *CLUSTER_ID* is the NFS cluster ID.

  This command disables the bandwidth control in the cluster. If cluster-level QoS is disabled, then share-level QoS is not applicable, even if it is enabled.

## Enabling input/output operations rate-limiting for NFS cluster

Learn how to enable the input/output operations (IOPS) rate-limiting for NFS cluster by using the command-line.

You can set input/output operations (IOPS) limits either per client or per share to ensure an even distribution of workload across the cluster.

> **Note:** The limit for IOPS rate-limiting is a minimum of 10 IOPS and a maximum of 16384 IOPS.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.
- An NFS cluster.
- Run the following command to enable bandwidth rate-limiting for an NFS cluster.

  ```
  ceph nfs cluster qos enable ops_control CLUSTER_ID qos_type:PerShare|PerClient|
  PerShare_PerClient [max_export_iops VALUE] [max_client_iops VALUE]
  ```

  Where,

  - *CLUSTER_ID* is the NFS cluster ID.
  - **qos_type** is the type of ops control. It can be **PerShare**, **PerClient** or **PerShare_PerClient**.
  - - If **PerShare qos_type** is selected, then the cluster-level QoS configuration applies to all shares created on that NFS cluster. It requires the **export_ops_count** parameter.

    - If **PerClient qos_type** is selected, then the cluster-level QoS configuration applies to all clients accessing shares created on that cluster. It requires the **client_ops_count** parameter.
    - If **PerShare_PerClient qos_type** is selected, then the cluster-level configuration applies to all shares as well as all clients of that NFS cluster. It requires both **export_ops_count** and **client_ops_count** parameters.
- **max_export_iops** is the count of IOPS operations for a share.
- **max_client_iops** is the count of IOPS operations for a client of a share.

> **Note:** If this command is used to update the **qos_type**, make sure to update all the shares with the required parameters.

**Example of IOPS rate-limiting for PerShare_PerClient, clust_nfs is the NFS cluster ID**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable ops_control clust_nfs
PerShare_PerClient --max_export_iops 2000 --max_client_iops 3000
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": true,
  "enable_bw_control": true,
  "enable_iops_control": true,
  "enable_qos": true,
  "max_client_combined_bw": "200.0MB",
  "max_client_iops": 3000,
  "max_export_combined_bw": "200.0MB",
  "max_export_iops": 2000,
  "qos_type": "PerShare_PerClient"
}
```

**Example of IOPS rate-limiting for PerShare**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable ops_control clust_nfs
PerShare --max_export_iops 4000
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": false,
  "enable_iops_control": true,
  "enable_qos": true,
  "max_export_iops": 4000,
  "qos_type": "PerShare"
}
```

**Example of IOPS rate-limiting for PerClient**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable ops_control clust_nfs
PerClient --max_client_iops 4000
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": false,
  "enable_iops_control": true,
  "enable_qos": true,
  "max_client_iops": 4000,
  "qos_type": "PerClient"
}
```

## Disabling input/output operations rate-limiting for NFS cluster

Learn to disable input/output operations (IOPS) rate-limiting for NFS clusters by using the command line.

– You can disable IOPS rate-limiting for the NFS cluster by using the following command.

```
nfs cluster qos disable ops_control CLUSTER_ID
```

Where,
*CLUSTER_ID* is the NFS cluster ID.

# Listing QoS configuration for NFS cluster

Learn how to list the quality of service (QoS) configuration for the NFS cluster by using the command line.

You can list the QoS configuration for the NFS clusters.

- Run the following command to list the QoS configuration:

```
ceph nfs cluster qos get CLUSTER_ID [--format VALUE]
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- *VALUE* is either YAML or JSON.

> **Note:** This command displays all the QoS global default configurations for a given cluster.

**Example of listing QoS settings with JSON format**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs --format json
{
"combined_rw_bw_control": false,
"enable_bw_control": false,
"enable_iops_control": true,
"enable_qos": true,
"max_client_iops": 1000,
"qos_type": "PerClient"
}
```

**Example of listing QoS settings with YAML format**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs --format yaml
combined_rw_bw_control: false
enable_bw_control: false
enable_iops_control: true
enable_qos: true
max_client_iops: 1000
qos_type: PerClient
```

# Managing rate-limiting for NFS shares

Learn to manage the rate-limiting or quality of service (QoS) for NFS shares.

There are 2 types of rate-limiting for NFS shares:

- Bandwidth rate-limiting.
- Input/output (IOPS) rate-limiting.

You can enable and disable the rate-limiting feature by using the command line.

## Enabling bandwidth rate-limiting for NFS shares

Learn how to enable bandwidth rate-limiting for NFS shares by using the command line.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.
- NFS shares.

**Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

Enabling bandwidth rate-limiting for NFS shares helps control network traffic by restricting data transfer rate for NFS clients. Bandwidth rate-limiting prevents network congestion, ensures fair bandwidth distribution, and optimizes system performance.

**Note:**

- When you set NFS QoS bandwidth control to a very low limit, other operations on the same file may experience delays. For example, if the QoS bandwidth is limited to 10 MB/sec and you execute the **du -h FILE** command while the file is under active write I/O, the command may take longer to complete.
- This behavior is expected because the file is servicing throttled write operations, which can delay concurrent access.
- You cannot enable share-level bandwidth control if the cluster-level **qos_type** is **PerClient**.

- The following command enables and updates QoS bandwidth control for shares.
  To enable share-level bandwidth control, you must enable cluster-level bandwidth control with **qos_type PerShare** or **Pershare_PerClient**. This command creates QOS_BLOCK in the share block.

```
nfs export qos enable bandwidth_control CLUSTER_ID <pseudo_path> [--combined-rw-bw-
ctrl] [--max_export_write_bw VALUE] [--max_export_read_bw VALUE] [--max_client_write_bw
VALUE] [--max_client_read_bw VALUE] [--max_export_combined_bw VALUE] [--
max_client_combined_bw VALUE]
```

  Where,

  - *CLUSTER_ID* is the NFS cluster ID.
  - **pseudo_path** is the pseudo-root path. (The path must be an absolute path.)
  - **combined-rw-bw-ctrl** enables combined read and write bandwidth control. When set to True, only the **max_export_combined_bw** and **max_client_combined_bw** parameters are allowed, depending on the qos_type.
  - **max_export_write_bw** is the maximum bandwidth that can be used for share write per second.
  - **max_export_read_bw** is the maximum bandwidth that can be used for share that is read per second.
  - **max_client_write_bw** is the maximum bandwidth that the client can use for writing per second.
  - **max_client_read_bw** is the maximum bandwidth that the client can use for reading per second.
  - **max_export_combined_bw** is the maximum combined read/write bandwidth for share per second.
  - **max_client_combined_bw** is the maximum combined read/write bandwidth for client per second.

  Example for bandwidth rate-limiting for QoS type **Pershare_PerClient**

```
[ceph: root@ceph-node-0 /]# ceph nfs export qos enable bandwidth_control clust_nfs /
export1 --max_export_write_bw 100MB --max_export_read_bw 200MB --max_client_write_bw
300MB --max_client_read_bw 100MB
[ceph: root@ceph-node-0 /]# ceph nfs export qos get clust_nfs /export1
{
  "cluster_enable_bw_control": true,
  "cluster_enable_iops_control": false,
  "cluster_enable_qos": true,
  "combined_rw_bw_control": false,
  "enable_bw_control": true,
  "enable_qos": true,
  "max_client_read_bw": "100.0MB",
  "max_client_write_bw": "300.0MB",
  "max_export_read_bw": "200.0MB",
  "max_export_write_bw": "100.0MB"
}
```

## Disabling bandwidth rate-limiting for NFS share

Learn how to disable bandwidth rate-limiting for NFS shares by using the command line.

> **Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

- You can disable bandwidth rate-limiting for the NFS shares by using the following command.

  ```
  nfs export qos disable bandwidth_control CLUSTER_ID PSEUDO_PATH
  ```

  Where,

  - *CLUSTER_ID* is the NFS cluster ID.
  - *PSEUDO_PATH* is the pseudo-root path. (The path must be an absolute path.)

  This command disables QoS bandwidth control for shares, allowing them to use unlimited input/output bandwidth. Once share QoS is disabled, the shares will no longer follow the bandwidth values set at the cluster level.

  > **Note:** To use cluster level QoS bandwidth control values for share again, you can use the NFS share apply *CLUSTER_ID* command, with the share section not having QOS_BLOCK.

## Enabling input/output operations rate-limiting for NFS share

Learn how to enable input/output operations (IOPS) rate-limiting for NFS share by using the command line.

Enabling IOPS rate-limiting for NFS shares helps control the number of input/output operations per second that clients can perform, preventing resource contention and ensuring fair usage.

To enable share-level IOPS control, you must enable cluster-level IOPS control with **`qos_type`** ,**`PerShare`**, or **`PerShare_PerClient`**.

> **Note:** You cannot enable export-level IOPS control if the cluster-level **`qos_type`** is **`PerClient`**.

**Understanding cluster-wide enforcement of bandwidth and IOPS limits**
When you enable bandwidth or input/output operations (IOPS) control on an NFS cluster, the configured limits apply cumulatively across all hosts in the cluster, rather than per individual host.

To enforce these cumulative limits, NFS services coordinate with NFS daemons running on other hosts in the cluster. This coordination ensures that bandwidth and IOPS limits are enforced consistently, even when multiple NFS servers handle client requests at the same time.

> **Note:** You can configure the port used for this inter-node communication by using the cluster_qos_port field in the NFS specification. The default port is 31311.

You can control how frequently NFS daemons exchange coordination messages by using the cqos_msg_interval field in the NFS specification. The default interval is 100 ms.

Alternatively, you can update the message interval by using the command-line interface.

```
ceph nfs cluster qos set <cluster_id> <msg_interval>
```

This inter-node coordination ensures accurate enforcement of bandwidth and IOPS limits across the NFS cluster.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.
- An NFS cluster.
- NFS shares.
- You can use the following command to enable IOPS rate-limiting to NFS shares. This command enables IOPS control for a specified share. You can also use this command to update the operations count.

```
nfs export qos enable ops_control CLUSTER_ID PSEUDO_PATH [max_export_iops:INT]
[max_client_iops:INT]
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- *PSEUDO_PATH* is the pseudo root path. The path must be an absolute path.
- **max_export_iops** is the count of IOPS operations for share.
- **max_client_iops** is the count of IOPS operations for a client of the share.

**IOPS rate-limiting for PerShare**

```
[ceph: root@ceph-node-0 /]# ceph nfs export qos enable ops_control clust_nfs /
export1 --max_export_iops 2000
[ceph: root@ceph-node-0 /]# ceph nfs export qos get clust_nfs /export1
{
"cluster_enable_bw_control": false,
"cluster_enable_iops_control": true,
"cluster_enable_qos": true,
"combined_rw_bw_control": false,
"enable_bw_control": false,
"enable_iops_control": true,
"enable_qos": true,
"max_export_iops": 2000
}
```

**IOPS rate-limiting for PerShare_Per Client**

```
[ceph: root@ceph-node-0 /]# ceph nfs export qos enable ops_control clust_nfs /
export1 --max_export_iops 2000 --max_client_iops 1000
[ceph: root@ceph-node-0 /]# ceph nfs export qos get clust_nfs /export1
{
"cluster_enable_bw_control": false,
"cluster_enable_iops_control": true,
"cluster_enable_qos": true,
"combined_rw_bw_control": false,
"enable_bw_control": false,
"enable_iops_control": true,
"enable_qos": true,
"max_client_iops": 1000,
"max_export_iops": 2000
}
```

## Disabling input/output operations rate-limiting for NFS share

Learn to disable input/output operations (IOPS) rate-limiting for NFS shares by using command line.

**Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

- The following command disables QoS operations control for sharing, removing any limits on the operations for shares. As a result, shares will not be subject to any cluster-level operation restrictions.

```
nfs export qos disable ops_control CLUSTER_ID PSEUDO_PATH
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- *PSEUDO_PATH* is the pseudo-root path. The path must be an absolute path.

## Listing QoS configuration for NFS share

Learn to list the quality of service (QoS) configuration for NFS shares by using the command line.

You can list the quality of service (QoS) configuration for the NFS shares.

> **Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

– Run the **`nfs export qos get`** command.

```
nfs export qos get CLUSTER_ID PSEUDO_PATH [--format VALUE]
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- *PSEUDO_PATH* is the pseudo-root path. (The path must be an absolute path.)
- *VALUE* is either JSON or YAML.

Example for QoS listing:

**QoS listing with format as JSON**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs --format json
{
"combined_rw_bw_control": false,
"enable_bw_control": false,
"enable_iops_control": true,
"enable_qos": true,
"max_client_iops": 1000,
"qos_type": "PerClient"
}
```

**QoS listing with format as YAML**

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs --format yaml
combined_rw_bw_control: false
enable_bw_control: false
enable_iops_control: true
enable_qos: true
max_client_iops: 1000
qos_type: PerClient
```

**QoS listing for a share**

```
[ceph: root@ceph-node-0 /]# ceph nfs export qos get clust_nfs /export1
{
"cluster_enable_bw_control": true,
"cluster_enable_iops_control": true,
"cluster_enable_qos": true,
"combined_rw_bw_control": false,
"enable_bw_control": true,
"enable_iops_control": true,
"enable_qos": true,
"max_client_iops": 1000,
"max_client_read_bw": "100.0MB",
"max_client_write_bw": "100.0MB",
"max_export_iops": 2000,
"max_export_read_bw": "200.0MB",
"max_export_write_bw": "100.0MB"
}
```

The share will not have any bandwidth restriction in this case; that is, it will not have any cluster level bandwidth restrictions.

## TLS security in NFS

Transport Layer Security (TLS), when used with Network File System (NFS), enables secure, encrypted communication for NFS traffic between the client and server.

TLS is a cryptographic protocol that provides confidentiality, integrity, and authentication for network communications. When used with NFS, TLS encrypts all traffic between the client and server. This ensures that data is not visible to intermediaries and cannot be altered in transit without detection.

TLS runs above Transmission Control Protocol (TCP) at the transport layer and establishes a secure channel between the client and server before any application data is exchanged.

A TLS handshake is the connection establishment phase. During the TLS handshake:

- The client validates the server's identity using its certificate.
- The server may optionally request a certificate from the client.

> **Note:** When NFS is configured to use TLS-based transport security, the mount negotiation and all subsequent Remote Procedure Call (RPC) operations occur over this encrypted channel.

### TLS versus mTLS

The authentication models supported by TLS are TLS and mTLS.

**TLS (one-way authentication)**

In TLS authentication, only the server is authenticated. The client verifies the server's certificate against a trusted Certificate Authority (CA). The server does not verify the client's identity beyond network address and application-level authentication.

**mTLS (mutual authentication)**

In mTLS authentication, both endpoints present certificates during the TLS handshake. The client validates the server's certificate, and the server validates the client's certificate against its trusted CA list. This ensures only authorized clients can connect.

### Use Cases

This feature is essential in scenarios where secure and compliant NFS communication is required, such as:

- SAP workloads requiring secure NFS communication.
- Migration from NetApp file services to Ceph-based NFS Regional File Services.
- Environments where compliance mandates encrypted data in transit.

## Configuring share security options for NFS

Set share security (XprtSec) options in NFS to control how clients connect using TLS or mTLS and understand the expected behavior for different configurations.

Share security options determine whether NFS traffic uses plaintext or encrypted (TLS or mTLS) . These settings affect client mount behavior and enforce encryption policies for secure communication.

- Share security options.

```
ceph nfs export create cephfs mynfs /export3 myfs --xprtsec tls
```

The expected behavior based on server share security and client mount options are listed in Table 1.

| *Share options and behavior* | | |
|---|---|---|
| **Server share security** | **Client mount option** | **Expected behavior** |
| XprtSec=none | (no xprtsec) | Mount succeeds in plaintext (unencrypted TCP/UDP). |

| Server share security | Client mount option | Expected behavior |
|---|---|---|
| XprtSec=none | xprtsec=tls | Mount succeeds over TLS; encryption used even though not required. |
| XprtSec=none | xprtsec=mtls | Mount succeeds over TLS; encryption used even though not required. |
| XprtSec=tls | (no xprtsec) | Mount fails with EACCES ("Permission denied") - server rejects non-TLS connections. |
| XprtSec=tls | xprtsec=tls | Mount succeeds over TLS; server certificate validated by client. |
| XprtSec=tls | xprtsec=mtls | Mount succeeds over TLS; client certificate validation is not enforced. |
| XprtSec=mtls | (no xprtsec) | Mount fails with EACCES - client not attempting mTLS. |
| XprtSec=mtls | xprtsec=tls | Mount fails with EACCES - mTLS required; client must provide valid certificate. |
| XprtSec=mtls | xprtsec=mtls | Mount succeeds only if client presents a valid certificate trusted by the server; otherwise fails with EACCES. |

You can specify share security options manually. For more information, see Manually specify share security options for NFS.

## Configuring TLS settings for NFS

Enable and customize TLS settings in NFS to secure NFS traffic with encryption and optional mutual authentication.

TLS settings in NFS control how encrypted communication is established between clients and the server. When you deploy the NFS service using this specification, certificates are generated and configured automatically based on the selected certificate source. The TLS settings also define the cipher suites, minimum protocol version, and optional features such as Kernel TLS (KTLS) and debugging.

- Deploy or update the NFS service specification file with TLS and SSL configuration.
  The following example shows an NFS service specification with TLS and SSL configuration by using inline certificates.

```
.. code-block:: yaml

    service_type: nfs
    service_id: mynfs
    placement:
      hosts:
      - ceph-node-0
    spec:
      port: 12345
      ssl: true
      certificate_source: inline|reference|cephadm-signed
      ssl_cert: |
        -----BEGIN CERTIFICATE-----
        (PEM cert contents here)
        -----END CERTIFICATE-----
      ssl_key: |
        -----BEGIN PRIVATE KEY-----
        (PEM key contents here)
        -----END PRIVATE KEY-----
      ssl_ca_cert:
        -----BEGIN PRIVATE KEY-----
        (PEM key contents here)
        -----END PRIVATE KEY-----
```

```
    tls_ktls: true
    tls_debug: true
    tls_min_version: TLSv1.3
    tls_ciphers: AES-256
```

TLS configuration parameters and their default values are listed in Table 1.

*NFS TLS parameters (inline certificate example)*

| Parameter | Default | Description |
|---|---|---|
| service_type | nfs | Defines the service type to deploy. |
| service_id | mynfs | Identifier for the NFS service instance. |
| placement | "" | Defines where the NFS service is deployed, including host selection and scheduling constraints. |
| hosts | ceph-node-0 | Host (or hosts) where the NFS service runs. |
| port | 12345 | TCP port that the NFS service listens on. |
| ssl | true | Enable or disable TLS for the NFS service. |
| certificate_source | inline \| reference \| cephadm-signed | Location of certificate material: inline content, external reference, or cephadm-signed. |
| ssl_cert | (PEM certificate) | Server certificate in PEM format (used when **`certificate_source`** is `inline`). |
| ssl_key | (PEM private key) | Server private key in PEM format matching **`ssl_cert`**. |
| ssl_ca_cert | (PEM CA certificate) | CA certificate (or chain) in PEM format used to validate the server certificate. |
| tls_ktls | true | Enable Kernel TLS acceleration if supported by the platform. |
| tls_debug | true | Enable detailed TLS debugging logs (use only for troubleshooting). |
| tls_min_version | TLSv1.3 | Minimum supported TLS protocol version. |
| tls_ciphers | AES-256 | Cipher suite policy for TLS connections. |

### Result

In case of configuration failure, you can configure TLS settings for NFS manually. For more information, see Manually configuring TLS settings for NFS.

## Setting up TLS in NFS client using Stunnel

Learn how to configure TLS for NFS to protect data in transit and ensure secure communication between NFS clients and servers.

### PREREQUISITE

Before you begin, make sure that you have the following prerequisites in place:

- Install TLS backend library (OpenSSL) based on your build configuration.
- The TLS certificates and keys must be available.

  > **Note:** You must have access to the following TLS certificates:
  > - CA certificate (`ca.crt`)
  > - Server certificate (`ganesha.crt`)
  > - Private key (`ganesha.key`)

- Ensure client-side support for TLS or mTLS (NFS client tools that support `xprtsec=tls` or `xprtsec=mtls`). For more information on configuring the NFS client with TLS or mutual TLS (mTLS) support **How to › Enhancing security › Data encryption in transit > Establishing encryption in transit for regional file share. › Establishing encryption in transit for regional file share** in Virtual Private Cloud (VPC) documentation in IBM Cloud Docs.

TLS adds an encryption layer to NFS traffic, safeguarding sensitive information from interception or tampering during transmission. By enabling TLS in NFS, you can enforce secure connections, maintain data integrity, and comply with security best practices.

Use Stunnel to enable TLS in older clients without native TLS support.

1. Install Stunnel on the client side.
2. Configure Stunnel (`/etc/stunnel/stunnel.conf`) to forward traffic to the NFS port.
   For example,

   ```
   pid = /var/run/stunnel/stunnel.pid
   cert = /home/dpp/workspace/TLS_certificates/vm1.crt
   key = /home/dpp/workspace/TLS_certificates/vm1.key
   CAfile = /home/dpp/workspace/TLS_certificates/ca.crt
   socket = r:TCP_NODELAY=1
   foreground = yes
   debug = 7
   output = /var/log/stunnel.log
   renegotiation = no
   [nfs4]
   client = yes
   accept = 127.0.0.1:49152
   connect = 23.13.161.10:2049
   sslVersion = TLSv1.3
   ciphers = ALL
   verify = 2
   ```

3. Change the `verify` value for TLS or mTLS as needed.

   > **Note:** Optional: Remove the `cert` and `key` entries if mTLS is not required on the client.

4. Start Stunnel.

   ```
   stunnel /etc/stunnel/stunnel.conf
   ```

5. Mount NFS through Stunnel.

   ```
   mount -vv -t nfs -o rw -o port=49152 127.0.0.1:/exportfs0 /mnt/
   ```

   You can enable TLS using tlshd (ktls-utils). For more information, see Setting up TLS in NFS clients using tlshd (ktls-utils).